

Wypożyczalnia Sprzętów Wodnych

Dokumentacja Projektu i Opis Implementacji (README)

1. Krótki opis tematu aplikacji

Aplikacja to system obiektowy odwzorowujący działanie wypożyczalni sprzętu wodnego (np. skuterów, kajaków, łodzi elektrycznych). Umożliwia zarządzanie asortymentem, obsługę procesu rezerwacji i zwrotów przez klientów oraz śledzenie historii transakcji, w pełni demonstrując kluczowe paradygmaty programowania obiektowego (OOP).

2. Lista klas i ich odpowiedzialności

- **WaterEquipment** – abstrakcyjna klasa bazowa reprezentująca ogólne cechy i operacje wspólne dla każdego sprzętu wodnego (odpowiedzialność: zarządzanie stanem dostępności i podstawowymi parametrami).
- **JetSki** – klasa szczegółowa dziedzicząca po `WaterEquipment` (odpowiedzialność: reprezentowanie skuterów wodnych o określonej mocy silnika).
- **Kayak** – klasa szczegółowa dziedzicząca po `WaterEquipment` (odpowiedzialność: reprezentowanie kajaków o określonej liczbie miejsc).
- **ElectricBoat** – klasa szczegółowa dziedzicząca po `WaterEquipment` i implementująca interfejs `IElectricEquipment` (odpowiedzialność: reprezentowanie ekologicznych łodzi elektrycznych wymagających ładowania).
- **IElectricEquipment** – interfejs określający kontrakt dla urządzeń elektrycznych (odpowiedzialność: wymuszenie implementacji metody ładowania akumulatora).
- **User** – klasa opisująca klienta wypożyczalni (odpowiedzialność: przechowywanie unikalnych danych identyfikacyjnych oraz rejestru historii wypożyczeń).
- **Rental** – klasa powiązania reprezentująca pojedyncze zdarzenie wypożyczenia (odpowiedzialność: łączenie obiektu użytkownika ze sprzętem w określonym czasie).
- **RentalManager** – klasa kontrolna systemu (odpowiedzialność: realizacja procesów biznesowych – dodawanie sprzętu, wypożyczanie, przyjmowanie zwrotów, wyświetlanie stanu asortymentu).

3. Opis relacji między klasami

- **Dziedziczenie (Inheritance):** Klasy `JetSki`, `Kayak` oraz `ElectricBoat` dziedziczą bezpośrednio po klasie abstrakcyjnej `WaterEquipment`.
- **Kompozycja / Agregacja (Aggregation):** Klasa `RentalManager` zawiera kolekcje (`List<WaterEquipment>` oraz `List<Rental>`), zarządzając cyklem życia transakcji wewnątrz systemu.

- **Relacja powiązania przez parametr metody:** Metoda `ProcessRental` w klasie `RentalManager` przyjmuje obiekty klas `User` jako parametry wejściowe, umożliwiając interakcję pomiędzy niezależnymi encjami.
- **Implementacja interfejsu:** Klasa `ElectricBoat` realizuje kontrakt zdefiniowany przez interfejs `IElectricEquipment`.

4. Wskazanie czterech zasad OOP w projekcie

- **Enkapsulacja:** Pola w klasie `WaterEquipment` (np. `_id`, `_rentalRate`) są prywatne. Dostęp do nich i walidacja danych (np. blokowanie ujemnych stawek czy pustych identyfikatorów) odbywa się wyłącznie poprzez publiczne właściwości (`get/set`). Klasa `User` zabezpiecza pole `AlbumNumber` przy użyciu modyfikatora `private set`.
- **Dziedziczenie:** Wspólne właściwości, takie jak marka, model, stawka godzinowa oraz stan dostępności, zostały zaimplementowane w klasie nadrzędnej `WaterEquipment`. Klasy pochodne (`JetSki`, `Kayak`, `ElectricBoat`) rozszerzają ją o specyficzne właściwości bez powielania wspólnego kodu.
- **Polimorfizm:** Metoda abstrakcyjna `GetEquipmentType()` zdefiniowana w klasie nadrzędnej jest nadpisywana (`override`) w każdej klasie potomnej. Dzięki temu wywołanie tej samej metody w pętli dla listy obiektów typu `WaterEquipment` generuje dynamicznie różne, dopasowane do typu komunikaty wyjściowe.
- **Abstrakcja:** Ukryto szczegóły implementacyjne poprzez utworzenie klasy abstrakcyjnej `WaterEquipment`, która uniemożliwia bezpośrednie utworzenie nieskonkretyzowanego obiektu sprzętu. Dodatkowo interfejs `IElectricEquipment` definiuje czysty kontrakt w postaci metody `Charge()`, pozostawiając szczegóły procesu ładowania klasie realizującej ten interfejs.